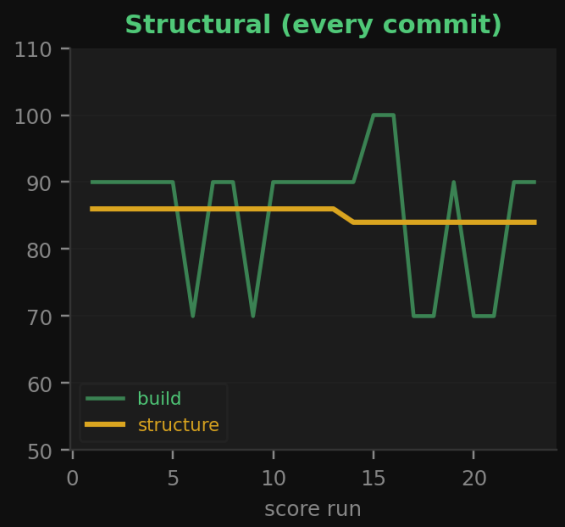
Automated evaluation loops for product development

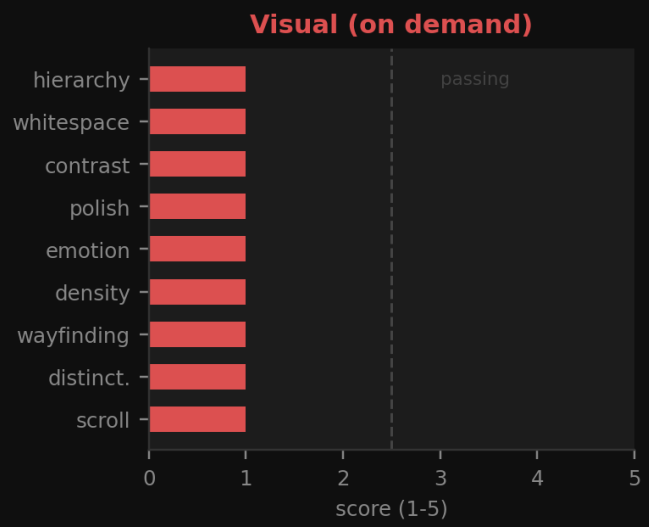
65 experiments. 2 projects. 2 days.
Real data, including the parts that don't work yet.

Jacob Rhinehart | March 2026

Two tiers: structural checks vs visual evaluation

Structural scoring (left) runs on every commit. Grep-based, fast, repeatable. Visual evaluation (right) takes Playwright screenshots and scores 9 taste dimensions via Claude vision. Both projects passed structural checks. Both scored 1/5 on taste -- routes were broken in production. That divergence is the entire point of running both.

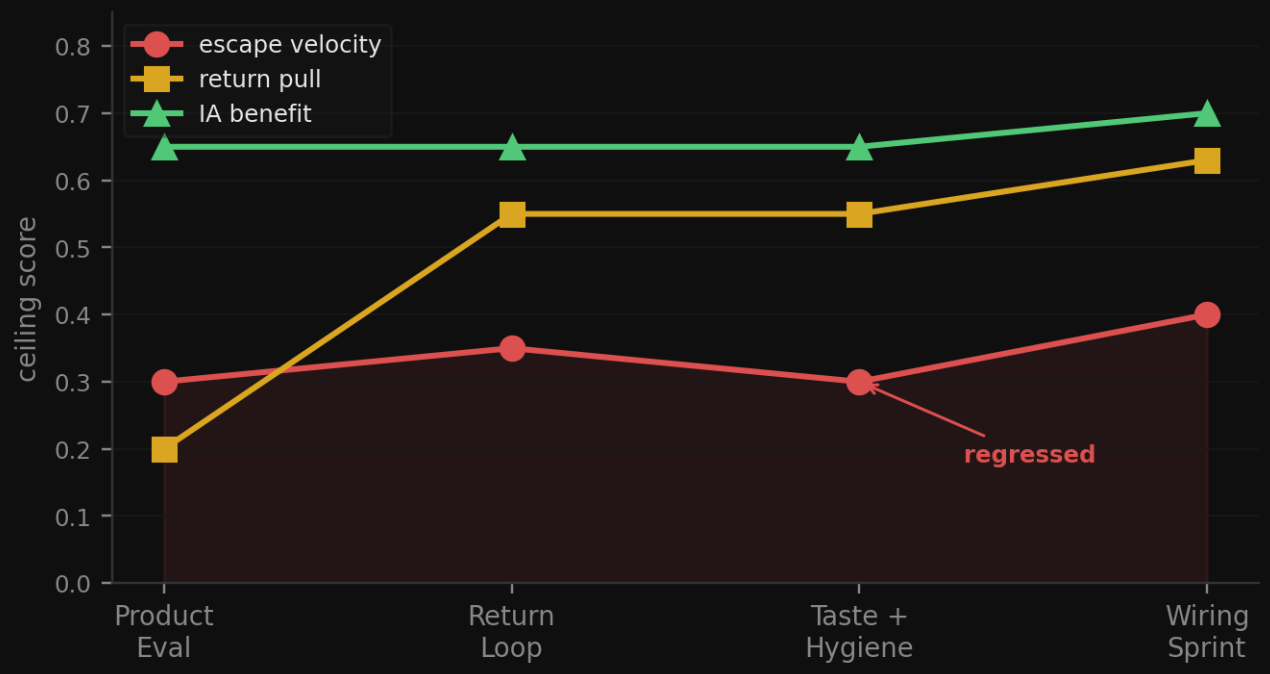




Caveat: taste score of 1/5 reflects deployment failures (404s), not design quality. This is a smoke test result, not a taste evaluation. The system caught it, but calling it "taste" overstates what was measured.

Ceiling scores across 4 evaluation cycles

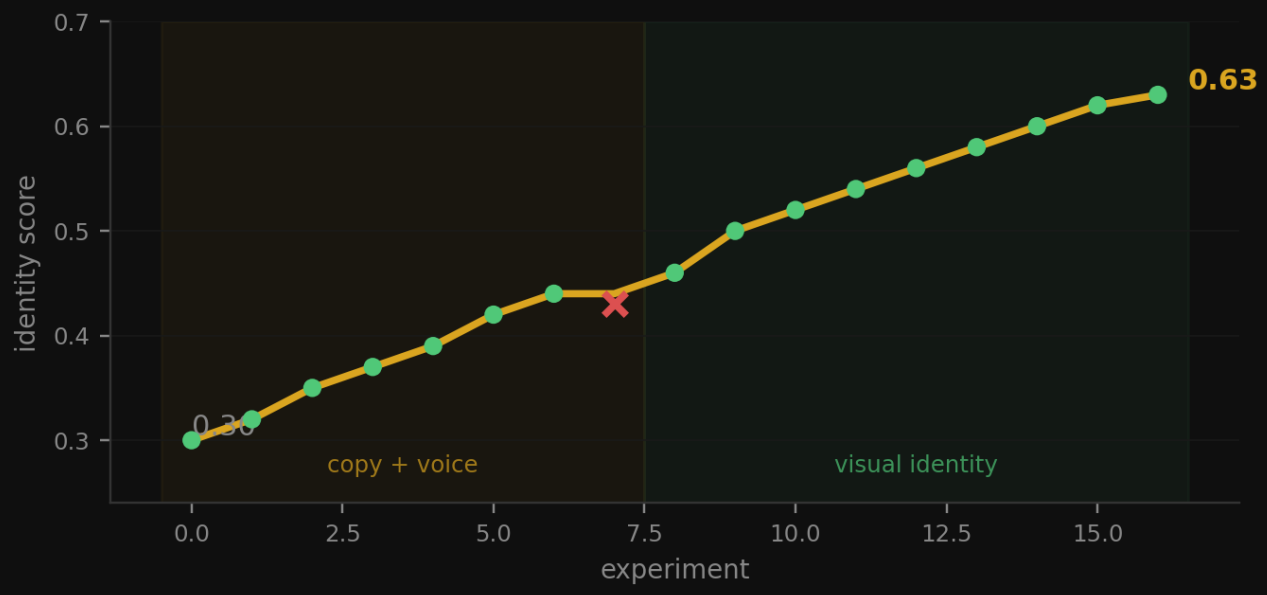
Return pull (0.20 to 0.63) improved significantly after targeted sprint work. Escape velocity is stubborn -- improved, regressed, then partially recovered. These ceiling scores are self-assessed benchmarks, not externally validated.



Caveat: these scores come from the system's own evaluation rubric. There is no external ground truth. The trend direction is more meaningful than the absolute numbers.

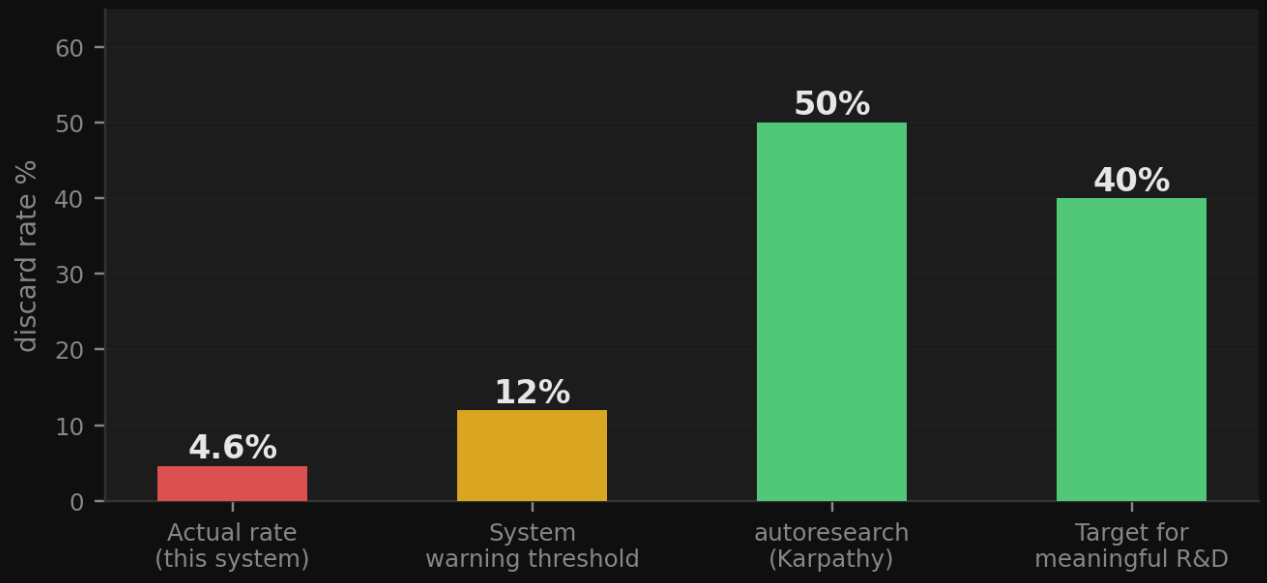
17 experiments, 1 discarded

Identity score went from 0.30 to 0.63 in one sprint. Copy and voice changes (Phase 1) plateaued around 0.44. Visual identity changes (Phase 2) pushed further. The one discard was a gold ping animation that improved the score but violated the product's constraint set.



The discard rate is too low

3 discards out of 65 experiments. 4.6%. For comparison, Karpathy's autoresearch discards roughly half of its runs. A low discard rate means the agent is making safe, incremental changes -- not testing real hypotheses. The system correctly flags this. Fixing it requires pushing the agent toward riskier experiments.



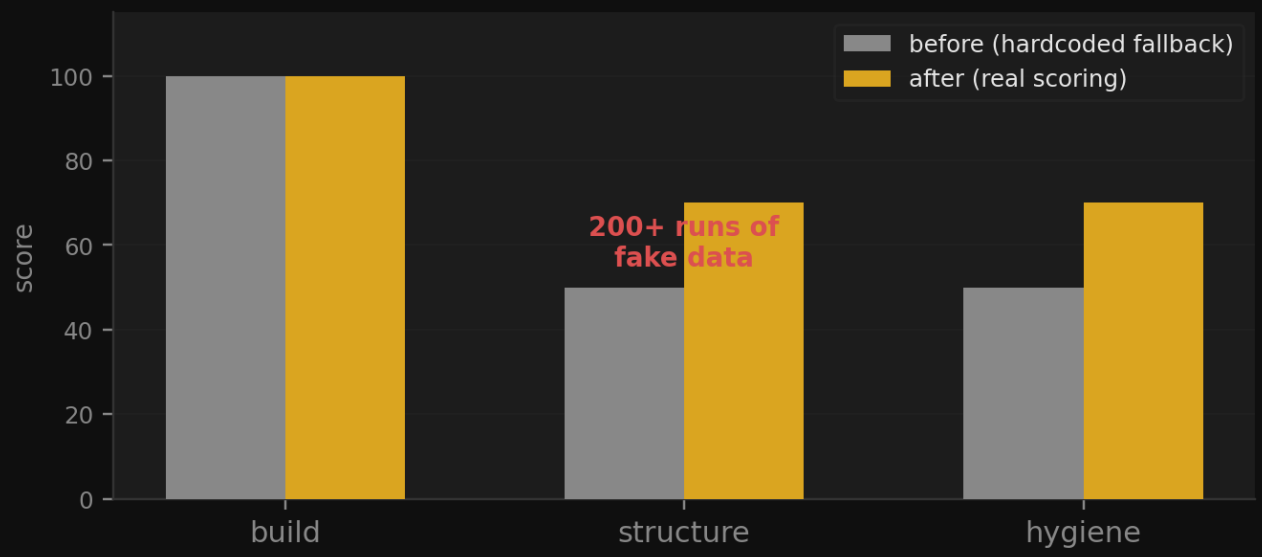
This is the most important slide. An experiment loop that rarely fails isn't experimenting -- it's committing.

Unresolved gaps persist until addressed

Each row is a problem surfaced by an evaluation. Red squares persist across cycles. Escape velocity has been flagged since eval 1 and remains partially open. Identity was resolved after a dedicated sprint. New problems emerge as old ones close. The mechanism is simple -- append-only gap tracking -- but it prevents backlog amnesia.

The system scored itself and found bugs

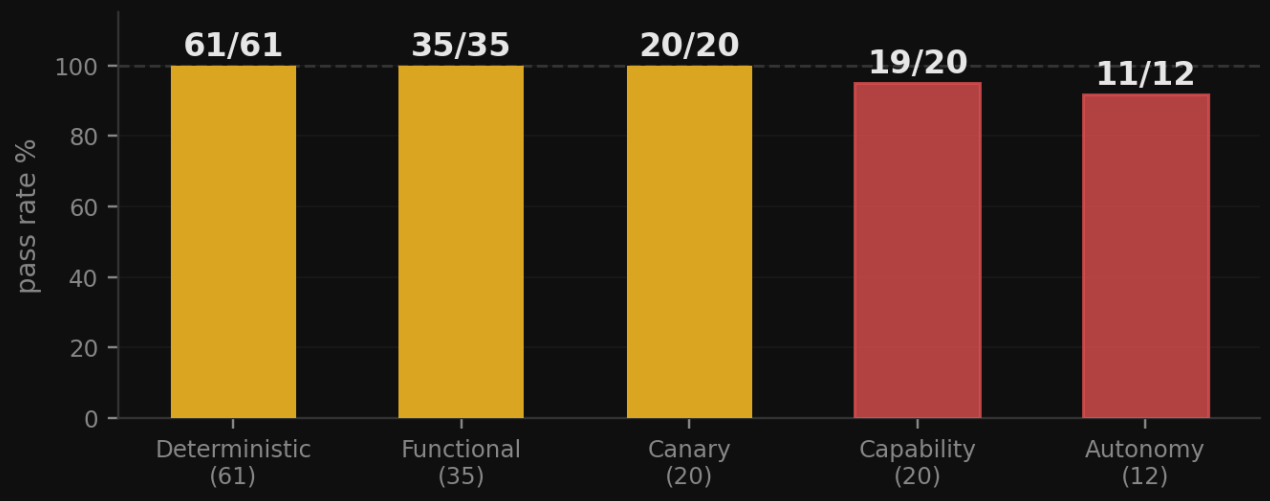
The scoring tool returned hardcoded 50/100 for 200+ runs because it didn't recognize CLI projects as scorable. The plateau detector was correctly flagging this the entire time. The warning existed. Nobody acted on it.



This is both the best and worst finding. Best: the system can catch its own failures. Worst: 200+ runs of decorative data means every prior conclusion drawn from those scores was unfounded.

141 / 143 self-eval tests

Five tiers of deterministic checks. No LLM-as-judge. The 2 failures: keep rate too high (slide 5) and taste score below MVP floor. The test suite warns at 100% pass rate because tests that always pass aren't testing anything.



All data from internal benchmarks. No external validation. N=2 projects, N=65 experiments, N=2 days. Interpret accordingly.